

- Databricks → it is a free app we will be using to code. The link will be update, under additional resources. (Search how does it work on youtube).

- Microsoft CERTIFICATE → you need to get to be certified, and build up your certificates.

- GITHUB → your data will be stored here, to build your portfolio.

- Canva → we will use it to build presentations, and Excel; theres canva, lovely and google sheets.

- Miro →

- Data studio → it is similar to power BI, you will learn how to build dashboards.

Google colab → machine learning codes for data sciences.

Draw.io → explains how data work / flow from one place to another, be able to draw as well.

o READ INTRO TO DATABASE !

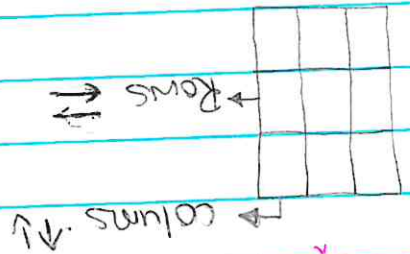
INTRODUCTION TO DATA.

DEFINE DATA:

- is a collection of facts, it is information. It is information that is collected to answer business questions. We use it to make business decisions.
- It can come in "different forms". Data is everywhere it comes in

Structured Data & Unstructured data

- refers to data stored in a format
- it has no specific format
- inconsistency format e.g. Music, podcast, Social media post, picture, video, a preacher, pdf document...



- When we have data we need to extract insights from the data. We need a tool to extract that data e.g. Spreadsheet, membership form, excel file
- Processes of Data collection.

Order number / Order pin

SQL

08.06.2026

Introduction to SQL

- ➔ **Structured Query Language (SQL)** is a scripting (or programming) language that we use to interact with structured data.

We use SQL to retrieve information (or data) from a structured table.

STUDENTS TABLE

Student ID	Name	Marks
101	Rohi	99
102	Mbali	25
103	Thabo	50
104	Lethabo	60

Select Statement - it allows us to select the specific columns from our table.

From Statement - allows us to specify the name of the table we are selecting the data from.

e.g. **SELECT** name, marks

FROM students - results

The select statement allows you to specify the columns you want in your final output.

Name	Marks
Rohi	99
Mbali	25
Thabo	50
Lethabo	60

→ **Select *** is a way to select all columns automatically from your table without specifying the column name one by one.

○ When you say **SELECT *** it is a way to get all the column names in that table.

○ Gets you all the column names from the table.
 From a table

○ When you are selecting specific column names, you separate each column name by a comma (,)

eg. **SELECT** Date, Gender, Age, Quantity
FROM retail-sales-dataset;

○ On the last column you specify, you do not put the comma.

means (different)

SELECT DISTINCT STATEMENT

○ We want to get different values from a specific column.

○ We remove duplicates / we don't repeat the same value.

○ **Syntax:** **SELECT DISTINCT** column-name **FROM** table-name;

What are the different products that are sold?

Specify the column name you want different values from.

Order ID	Products name	Revenue
101	Apple	100
102	Orange	50
103	Apple	60
104	Banana	150

← **FROM** fruits - orders.

SELECT DISTINCT prd-name

Prd name

Apple
Banana
Orange.

e.g. `SELECT DISTINCT 'Product Category'`
`FROM retail-sales-dataset;`

04.06.2026

DATA FILTERING

• We filter data if we want to get specific information from our data.

• It is a way to exclude row/information of data that do not meet a specific criteria.

• You want specific information.

• We filter based on conditions

WHERE statement

Syntax: `SELECT column1,`
`column2,`
`column3`
`FROM table-name`
`WHERE column condition;`

A filtered table is a reduced table.

Write a SQL query to give me only the names of students that obtained more than 60% in their exam.

Student Results

Student-ID	Name	Marks
101	Rohi	99
102	Mihali	85
103	Thabo	50
104	Leethabo	60

`SELECT *`
`FROM student-results`
`WHERE marks > 60;`



Answer in orange cloud.

```

SELECT *
FROM retail_sales_dataset
WHERE Gender = 'male' !

```

SQL OPERATORS

10.06.20

* Multiple Operators.

1 Arithmetic Operators.. we use them while coding.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division

2 Comparison Operators 3. Logical Operators

Operator	Description	Operator	Description
=	Equal to	AND	Both conditions must be met
>	Greater than	OR	1 condition must be met
<	Less than	Between	must be within the range
>=	Greater than or equal to	IN	must be contained in the list
<=	Less than or equal to	NOT	Gives the opposite result of what is specified.
<>	Not equal to	Like	Looks for a pattern

Operators - are tools you use in coding

DATA FILTERING

• We use the WHERE statement

• It is a filtering statement, that we use to filter out data based on specific conditions.

• We can use the SQL operators to build the conditions for filtering our data.

• Syntax: SELECT column1,

column2,
column3

FROM table-name
WHERE condition;

Users
Column name
Value

User ID	User name	City
1	Nomvula	Johannesburg
2	David	Cape Town
3	Anele	Durban
4	Kabelo	Pretoria
5	Lerato	Port Elizabeth

Write an SQL query to give me the name of users in Durban.

SELECT *
FROM users

WHERE city = 'Durban';
Condition - the condition.

User ID	User name	City
3	Anele	Durban

Write SQL query

to give me the

Monthly plan that

cost more than Rs 50

Membership plan

Plan-ID	Plan name	Monthly-price
101	Basic	79
102	Standard	129
103	Premium	199
104	Family	249
105	Mobile	59

SELECT plan-name

monthly-price

FROM membership-plan

WHERE monthly-price > 150

Plan-name	Monthly-price
Premium	199
Family	249

Sorting Statement

- a way to sort our data we use the **ORDER BY** statement. In SQL this is a sorting statement.

- We can sort our data either in ascending order (ASC) or descending order (DESC)

- Sorting does not remove or add anything to new our data - it just re-arrange the rows in our data.

Ascending order - it is going up, from smallest to biggest, from lowest to highest (ASC)

Descending order - it is going down, from biggest to smallest, from highest to lowest.

Syntax : **SELECT ***

FROM table-name
WHERE condition
ORDER BY column-name ASC;
 Ascending

SELECT *
FROM table-name
WHERE condition
ORDER BY column-name DESC;
 Descending

- is used to perform a calculation on rows of a specific column and returns a single value
- Single value per group.

Aggregate functions.

John	25	John	25
Musa	23	Lerat	20
Thato	22	Thato	22
Lerato	20	Musa	23
Sarah	19	John	25
Name	Age	Name	Age

SELECT name, age
FROM students
ORDER BY age ASC;

SELECT name, age
FROM students
ORDER BY age DESC;

Student ID	Name	Course	Age	City
1	Thabo	Data analysis	22	Johannesburg
2	Sarah	Data analysis	19	Pretoria
3	John	Data analysis	25	Durban
4	Lerato	Data analysis	20	Johannesburg
5	Musa	Data analysis	23	Johannesburg

FUNCTION	DESCRIPTION
COUNT()	Counts the number of rows.
SUM()	Calculates the total of numeric column.
AVG()	Calculates the average of a numeric column.
MAX()	find the maximum value
MIN()	find the minimum value.

e.g. Employees.

Emp-ID	Name	Salary
101	John	30
102	Mary	40
103	David	35

SELECT min (salary) AS min-salary FROM employees;

Min-salary
30

is a way to SELECT max (salary) AS max-salary FROM employees; after applying functions.

Max-salary
40

We are doing a count

17.06.2026.

(Continue from yesterday.
e.g. GRADE 10 H

Students

Student_id	Name	Subject
101	Rafhi	Science
102	Jeff	Science
103	Mark	Commerce
104	Elon	Science
105	Steve	History
106	Donald	Commerce

SELECT COUNT (DISTINCT student_id) AS number-of-students
 FROM students;

counting the number of students in the class using their ID.

Number of Students	6
--------------------	---

- We count something that is uniquely identified like an ID
- We do not count the revenue, we sum() the revenue.
- We do not sum an ID, rather we count an ID

GROUP BY Statement

IS A Statement that allows us to group our data into categories

SELECT subject1
 FROM students
 GROUP BY subject

count (DISTINCT student_id) as number-of-students

FROM students
GROUP BY subject

- we group by a column that is not aggregates in the select statement
- we commonly group by a categorical column.

Subject	Number-of-students
Science	2
Commerce	3
History	1

Sale-id	Customer-name	Category	Revenue
1	John	Electronics	120
2	Mary	Food	80
3	Elen	Electronics	150
4	Peter	Clothing	100

Q. Write an SQL query to calculate the total revenue for each category.

SELECT SUM revenue, COUNT(DISTINCT-category)
 FROM Sales
 GROUP BY revenue
 As total-revenue
 FROM sales
 GROUP BY category;

Corrections.

Category	Revenue
Electronics	270
Food	80
Clothing	100

Aggregate Function

- Summarise data - Extract specific information.
- perform calculations on multiple rows of a specific column and returns a single value / single value per group
- Consolidate data

1. COUNT() - we count unique identifiers (ID)
2. SUM() - We sum numeric data (revenue)

- AVG() - "
- MIN() - Looks for min value in a column
- MAX() - Looks for max value in a column.

GROUP BY statement

- SELECT categorical - column, gender, rate, category
- aggregate-function (column) AS descriptive column

FROM table - name

GROUP BY categorical - column;

↳ we group by a column that is not aggregated

NEW STATEMENT

HAVING Statement

- it is used to filter data that is aggregated
- we apply the having statement to data that is already aggregated.

- It is used to filter ~~used~~ grouped results after using a GROUP BY

WHERE filters rows before grouping

HAVING - filters groups after a Group By

Sales

Cust - id	Cust - name	Product	Amount
101	Rofhi	Laptop	R5000
101	Rofhi	mouse	R300
102	Abel	Keyboard	R700
102	Abel	Monitor	R2500
103	Clara	Mouse	R300

Write a SQL query to find customers who spent more than R3000

SELECT cust-name

SUM (amount) AS total_spent

FROM sales

GROUP BY cust-name

HAVING SUM (amount) > 3000

Customer name

Rofhi

Abel

Clara

R300

R3200

R5300

total spent

11

CASE STATEMENT

- Allows us to build / create a new column based on logical conditions applied on other columns.
- A case statement allows us to classify our numerical data into categorical groupings.

Syntax: `SELECT column1, CASE`

WHEN condition 1 THEN result 1
WHEN condition 2 THEN result 2
ELSE default = result
END AS column-name.
FROM table-name.

employees

Business Logic

emp-id	name	Salary
101	John	30000
102	Peter	7000
103	Samson	12000
104	David	4000
105	Goliyard!	800

Salary below 5000 → low
Salary between 5000 & 10000 → Med
Salary above 10000 → High

`SELECT name,`

- a case statement creates a new column, o the case statement is placed within the select statement.

CASE

WHEN salary < 5000 THEN 'Low'

WHEN salary BETWEEN 5000 AND 10000 THEN 'Med'

WHEN salary > 10000 THEN 'High'

END AS salary - bracket

FROM employees;

Output

name	salary	salary-bracket
John	3000	low
Peter	7000	Med
Samson	12000	high
David	4000	low
Galliyadi	300	low

SELECT product-name, price

CASE

WHEN price > 1000 THEN 'expensive'
 WHEN price BETWEEN 100 & 1000 THEN 'Mid-range'
 WHEN price < 100 THEN 'Budget'

END AS price-category
 FROM products

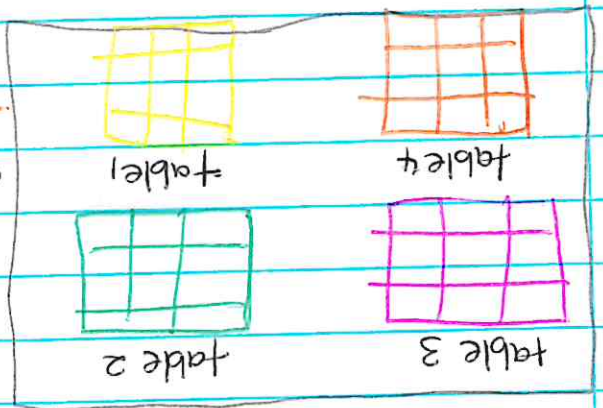
product-name	price	price-category
Laptop	1200.00	expensive
Phone	800.00	Mid-range
Keyboard	45.00	Budget
Monitor	300.00	Mid-range
Mouse	25.00	Budget

24.06.2026

Relational Database

DW - Data Warehouse

• A dataware have multiple dat



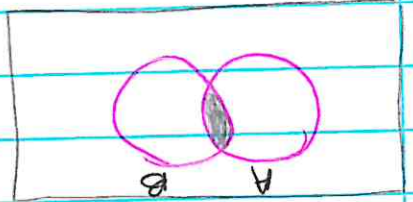
Data Management System (DMS)

We can join multiples into new

SQL Joins

An sql is meant to join multiple tables based on common columns.

INNER JOIN



- We join tables on columns but the join affect the output rows
- When we join table we can select any column between the tables we are joining
- When you join you can select anything from the two tables.
- Returns rows where there is a match on both the tables.

Syntax: `SELECT A.column1,`

`A.column2,`

`B.column3,`

`B.column4`

FROM table1 AS A
INNER JOIN table2 AS B

ON A.common-column = B.common-column

CUSTOMERS the cust id is different for each customer.

ORDERS

customer is a primary key in the customers table

Dimension table

Cust-id	name
1	Rofhi
2	Mballi

Order-id	Cust-id	Amount
101	1	500
102	4	20
103	5	10

SELECT cust-id, name, amount
FROM customers AS A
INNER JOIN orders AS B
ON A.cust-id = B.cust-id;

is a foreign key - is not unique
it can be repeated.

Cust-id	name	amount
1	Rofhi	500

the results will show what appear in both table only.

DISSCRIPTIVE TABLE.

Gives more information to the fact table.

Data stored in a Dim table does not change frequently.

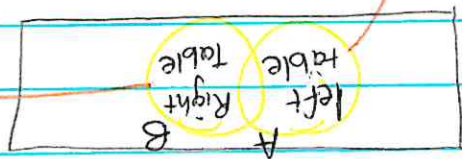
Informative table

fact table.

We use for analysis &

it processes information fast

LEFT JOIN



take information you need.

SELECT

column1,
column2,
column3

FROM

table1 AS A

LEFT JOIN

table2 AS B

ON A.column = B.column

Left table
Right table

- In our final outcome, we need all the rows in the left table
- Priority table, Base table
- main table, focus table

eg. A left join returns all records from the left table, and only the matching records from the right table.

When there is no match in the right table we still keep the left table row, but the right table columns will show NULL

Customers - table		Orders - table	
Cust-id	Cust-name	Order-id	Amount
1	Rohiniwa	101	500
2	Lerate	102	300
3	Mballi		

Write a SQL query to give a list of all customers & what they ordered.

NULL - is a empty space

select cust-id, amount, order-id

from customers table AS A

LEFT JOIN orders-table AS B

ON A.cust-id = B.cust-id

Cust-id	Cust-name	Amount	Order-id
1	Rohini	500	101
2	Lerate	300	102
3	Mballi	NULL	NULL

CASE STUDY

- Step 0: Reading & understanding the Business Requirements.
- Step 1: Asking data driven questions.

Users

- What age group are my highest subscriber?
- What gender are my subscribers? and what are they watching.

- Where are most of my subscribers coming from (Province)?
- What race are my subscribers?
- Can I personalise the campaigns
- When are the subscribers watching the content?
- What content has the highest views?
- What is their watch time?
- What time of day are they watching?

Viewership

Analysis

- Step 2: Data inspection → inject data on databricks - check available data

- Step 3: Data processing
- After inspecting data check what can be done on the data, this includes data cleaning, manipulation

Step 4:

User Id

Name

Surname

Email

Gender

Race

Age

Province

Social Media Handle

Window Functions

What are window functions in SQL
performs calculations across a set of table rows that are related to the current row.

(Core Syntax

SELECT column-name,
Window Function () OVER (

PARTITION BY partition-column
ORDER BY sort-column
) AS new-column

LAG Function

allows you to look at a previous row.
Previous Row (previous rows vs current row)

SQL

LAG (column-name) OVER (

ORDER BY column-name

LEAD Function

Next row
allows looks into the next row (current row vs next row)

YOU PARTITION BY SOMETHING THAT IS CATEGORICAL